Quant Finance Starter Projects (India & US)

Three small, self-contained projects that teach the core ideas of quantitative finance. Each is a single Python script you can read top to bottom.

```
pip install numpy pandas yfinance   # yfinance optional

# Indian markets (NSE) - the default
python 01_monte_carlo_var.py                       # RELIANCE, HDFCBANK, TCS, INFY, ITC; Rs 1 crore
python 02_pairs_trading.py                         # HDFCBANK vs ICICIBANK, costed as stock futures
python 03_ma_crossover.py                          # NIFTYBEES (Nifty 50 ETF)
python 03_ma_crossover.py --ticker RELIANCE.NS --instrument equity_delivery
python 04_ma_sweep.py --csv nifty.csv               # SMA vs EMA sweep, columns Date,Close
python markets.py                                  # print the Indian cost model

# US markets
python 02_pairs_trading.py --market us             # KO vs PEP
python 03_ma_crossover.py --market us              # SPY

# add --synthetic to any script to run offline on simulated data
```

NSE tickers use Yahoo's `.NS` suffix (BSE: `.BO`). Other pairs to try: `TCS.NS`/`INFY.NS`, `SBIN.NS`/`BANKBARODA.NS`, `HINDUNILVR.NS`/`DABUR.NS`, `TATASTEEL.NS`/`JSWSTEEL.NS`, `ULTRACEMCO.NS`/`AMBUJACEM.NS`.

If `yfinance` is missing or offline, the scripts fall back to synthetic data automatically.

---

1. Monte Carlo portfolio risk (`01_monte_carlo_var.py`)

Goal: How much could this portfolio lose in a bad month, and how bad does it get beyond that?

Steps

1. Download prices and compute daily log returns.
2. Estimate each stock's mean return and volatility, plus the correlation matrix.
3. Build the covariance matrix and take its **Cholesky decomposition** `L`, so that `L @ z` turns independent normal shocks `z` into correlated ones.
4. Simulate 10,000 paths of 21 trading days and compound each one into a one-month portfolio return.
5. **Value at Risk (95%)** = the 5th-percentile loss.
Expected Shortfall = the average loss in the worst 5% of outcomes.
6. Re-run with correlation forced to 0 and to 1.

What you learn: The volatilities stay the same in all three runs. Only correlation changes, and VaR moves a lot (in the sample run: $63k with zero correlation, $103k with the estimated correlation, $129k with perfect correlation). Diversification comes from correlation.

Extensions: fat tails (Student-t shocks), historical bootstrap instead of

normal shocks, backtest VaR by counting breaches, optimise weights to minimise ES.

---

2. Pairs trading / statistical arbitrage (`02_pairs_trading.py`)

Goal: Profit when two related stocks drift apart, then come back together.

Steps

1. Split the history into a **formation** period (60%) and a **trading** period (40%).
2. On formation data, regress `A = alpha + beta * B`. `beta` is the **hedge ratio**.
3. **Engle-Granger test:** run an ADF test on the regression residuals. A statistic below about -3.34 means the pair is cointegrated at 5%. (These critical values are stricter than ordinary ADF ones because beta was estimated.)
4. Spread = `A - beta*B - alpha`. Z-score = (spread - rolling mean) / rolling std.
5. If z > +2, short the spread (short A, long B). If z < -2, go long the spread. Exit when z crosses 0.
6. Lag positions by one day and charge costs on both legs.
7. Compare in-sample and out-of-sample results, and re-test cointegration on the new data.

What you learn: Mean reversion and hedging. The biggest lesson is that **relationships break**. A pair that passes the test in-sample can stop being cointegrated later, so always validate out of sample and use a stop-loss (for example, exit at |z| > 4).

Extensions: rolling or Kalman-filter hedge ratio, half-life of mean reversion (fit an AR(1) to the spread), scanning a sector for candidate pairs (watch out for multiple-testing bias).

---

3. Moving-average crossover backtest (`03_ma_crossover.py`)

Goal: Test the simplest strategy there is, and test it properly.

Rules: Hold the asset when the 50-day SMA is above the 200-day SMA. Otherwise hold cash.

Testing discipline

- **Delay signals by one day.** The signal is computed on today's close, so you can only trade on it tomorrow (`position = signal.shift(1)`). The script also prints a look-ahead version that skips this lag. The gap between the two is the kind of performance a bug creates.
- **Transaction costs** are charged on every position change (default 10 bps).
- **Benchmark against buy and hold**, comparing CAGR, volatility, Sharpe ratio and max drawdown.

What you learn: Most strategies fail at this stage. The crossover usually gives up return in exchange for a smaller drawdown. If a backtest looks too good, look for look-ahead bias, survivorship bias, missing costs, or parameters fitted to the same data you tested on.

Extensions: a parameter sweep (and then ask why the best parameters won't hold up), walk-forward testing, applying it to many assets.

3b. Which crossover is best? SMA vs EMA sweep (`04_ma_sweep.py`)

The script tests 70 rules (SMA and EMA, fast 1-100, slow 20-250) with a one-day lag, ETF costs, cash earning 6% and a 1.3% dividend yield added back to the price index. Then it picks the winner using data before 2013 and checks how that winner did after 2013.

Results on Nifty 50 daily data, 2002-2023:
• Buy and hold: 16.4% CAGR, Sharpe 0.54, max drawdown -59%.
• The best rules were short-term **price vs 50/100-day MA** and **EMA 20/50**, at 17-18% CAGR, Sharpe 0.75-0.85, and drawdowns of -18% to -31%.
• The famous **50/200 cross came last of the classics**: 12.1% CAGR, Sharpe 0.41.
• EMA vs SMA: EMA was slightly better on average (Sharpe 0.57 vs 0.54).
The choice of lookback matters far more than the type of average.
• Out of sample (2013-2023) **no rule beat buy and hold on return**: 14.3% vs about 11-13% for the best rules. What they kept was a much smaller drawdown (about -11% to -15% vs -38%).
• On 48 Nifty 50 stocks from 2021 to 2025, with delivery costs (32 bps round trip), every rule lost to buy and hold by about 7% a year on the median stock.

The "best crossover" changes from one period to the next. What a trend filter reliably gives you is protection in a crash. It doesn't add return.

Indian-market specifics

Transaction costs (`markets.py`, as of Nov 2026, after the Budget 2026 STT revision that took effect 1 April 2026; check them against your broker's charges page):

Instrument	STT	Stamp duty	NSE txn	Round trip*
Equity delivery	0.1% buy + 0.1% sell	0.015% buy	0.00297%	~32 bps
Equity ETF	0.001% sell	0.015% buy	0.00297%	~12 bps
Stock futures	0.05% sell	0.002% buy	0.00183%	~16 bps

*Includes 18% GST on exchange and SEBI fees, plus 5 bps of slippage per side. Brokerage defaults to 0; set `BROKERAGE` if your broker charges for it. Delivery trades in India cost roughly 3x what they do in the US, so strategies that trade

often get hit hard.

Things that differ from the US

• **No overnight shorting in the cash market.** Short-selling equity has to be closed out the same day. To carry a pairs trade overnight you need **stock futures** (only about 200 F&O stocks, fixed lot sizes, monthly expiry and rollover, a basis against spot, and F&O ban periods when open interest hits 95% of the market-wide limit) or SLB. The pairs script costs both legs as futures but doesn't model lot sizes, rollover or basis.
• **High risk-free rate (~6%).** Sharpe ratios here subtract it, and cash in the MA strategy earns it (turn this off with `--no-cash-yield`). A strategy making 8% a year is only barely beating T-bills.
• **The ^NSEI index excludes dividends**, so it understates buy-and-hold by about 1-1.5% a year. Backtest on a tradable ETF (NIFTYBEES) or the Nifty 50 TRI.
• **Circuit limits:** stocks outside F&O can lock at their price band, so you may not be able to exit at the price your backtest assumes.
• **Survivorship bias:** today's Nifty 50 constituents aren't the 2010 ones. Backtesting only current constituents overstates returns.
• **Taxes:** STCG of 20% and LTCG of 12.5% above Rs 1.25 lakh. Frequent trading turns long-term gains into short-term ones. Taxes aren't modelled here.
• **Liquidity:** stick to large caps. Mid and small caps have wider spreads than the 5 bps slippage assumption.

Checklist for any backtest

■ No future data: all signals lagged
■ Costs and slippage included
■ Benchmark comparison
■ Out-of-sample or walk-forward period
■ Parameters chosen before seeing test data
■ Survivorship-bias-free universe

Educational code, not investment advice.